

MORTAR: Multi-turn Metamorphic Testing for LLM-based Dialogue Systems

Guoxiang (Aaron) Guo, Aldeida Aleti, Neelofar Neelofar, Chakkrit Tantithamthavorn, *Senior Member, IEEE*,
Yuanyuan Qi, and Tsong Yueh Chen, *Fellow, IEEE*.

Abstract—With the widespread application of LLM-based dialogue systems in daily life, quality assurance has become more important than ever. Recent research has successfully introduced methods to identify unexpected behaviour in single-turn testing scenarios. However, multi-turn interaction is the common real-world usage of dialogue systems, yet testing methods for such interactions remain underexplored. This is largely due to the oracle problem in multi-turn testing, which continues to pose a significant challenge for dialogue system developers and researchers. In this paper, we propose MORTAR, a metamorphic multi-turn dialogue testing approach, which mitigates the test oracle problem in testing LLM-based dialogue systems. MORTAR formalises the multi-turn testing for dialogue systems, and automates the generation of question-answer dialogue test cases with multiple dialogue-level perturbations and metamorphic relations (MRs). The automated MR matching mechanism allows MORTAR more flexibility and efficiency in metamorphic testing. The proposed approach is fully automated without reliance on LLM judges. In testing six popular LLM-based dialogue systems, MORTAR reaches significantly better effectiveness with over 150% more bugs revealed per test case when compared to the single-turn metamorphic testing baseline. Regarding the quality of bugs, MORTAR reveals higher-quality bugs in terms of diversity, precision and uniqueness. MORTAR is expected to inspire more multi-turn testing approaches, and assist developers in evaluating the dialogue system performance more comprehensively with constrained test resources and budget.

Index Terms—Metamorphic Testing, Dialogue System Testing, SE4AI

I. INTRODUCTION

Rapid development of large language models (LLMs) has led to substantial capability improvements in downstream applications, particularly in dialogue systems [1]. A dialogue system is an interactive conversational system that leverages machine learning models such as LLMs to generate and understand human-like responses in natural language. From a system developer's perspective, comprehensive testing is essential to ensure the service quality of DS. However, testing such AI systems inevitably encounters the oracle problem [2]. In software testing, the test oracle is crucial, as it determines whether the system under test (SUT) behaves as expected. Traditionally, the development of test oracles relies on the

experience of system testers and the design of test suites. Crowdsourcing workers also play an important role in producing dialogue datasets as test cases. Several evaluation datasets, e.g. HotpotQA [3], DecodingTrust [4], etc., can be used to simulate user input and evaluate if the generated content matches common sense and human values. Some datasets, e.g. MMLU [5], GPQA [6], etc., offer high-quality test cases that require expert knowledge to answer. These datasets have successfully revealed defects in LLM-based dialogue systems during single-turn conversations with the simulated user.

Developers and researchers have proposed many testing methods to discover bugs in LLM-based systems. The form of existing testing can be categorized into single-turn and multi-turn. Single-turn testing involves presenting the LLM with a standalone input prompt and evaluating the quality of its immediate response. Multi-turn testing is conducted through multiple back-and-forth rounds, which simulate real-world dialogue between the user and LLM. It is worth noting that most existing testing datasets and approaches only provide single-turn testing capability [7]. According to real-world usage, however, over 63% of dialogues contain more than 2 rounds [8]. This gap between system testing and development can lead to critical issues in real-world usage, e.g. existing dialogue systems exhibit a higher likelihood of generating harmful content in multi-turn interactions [9].

The challenge of multi-turn testing roots in the automation of test oracles. Human judgements are golden, but are also inevitably costly and inaccessible in testing scenarios where time and resource are constrained. An increasing number of testing approaches rely on LLMs to act as a judge in multi-turn dialogue testing [10], [11], while LLM judges are generally biased and could harm the reliability of evaluation [12]. Considering the gap between existing approaches, which primarily focus on single-turn interactions, and the multi-turn nature of real-world systems, this research aims to propose a new automated approach for multi-turn dialogue testing without reliance on LLM judges.

Metamorphic testing (MT) [13], a well-established software testing method, has shown the potential to alleviate the oracle problem in the testing of generative AI (GenAI) systems [2]. Several MT-based approaches have been introduced to support single-turn testing for question-answering (QA) systems and large language models (LLMs) [14]–[16]. Perturbations in character level, word level, and sentence level show effectiveness in some testing scenarios. However, relying solely on single-turn perturbations in multi-turn dialogue testing fails to account for the essential characteristic of multi-turn inter-

G. Guo, A. Aleti, C. Tantithamthavorn and Y. Qi are with the Faculty of Information Technology, Monash University, Clayton, VIC 3800, Australia. (e-mail: Guoxiang.Guo, Aldeida.Aleti, Chakkrit, Yuanyuan.Qi@monash.edu)

Neelofar is with the School of Computing Technologies, RMIT University, Melbourne, VIC 3000, Australia (e-mail: neelofar.neelofar@rmit.edu.au).

T. Y. Chen is with the School of Science, Computing and Emerging Technologies, Swinburne University of Technology, Hawthorn, VIC 3122 Australia (e-mail: tychen@swin.edu.au).

Corresponding author: Aldeida Aleti (e-mail: Aldeida.Aleti@monash.edu).

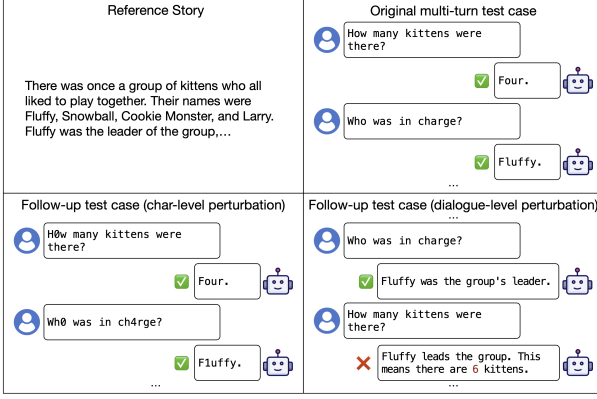


Fig. 1. Testing with different test cases. The test seed is the original test case with multiple question rounds. Dialogue-level perturbation (lower right) alters the context of questions, which enables follow-up test cases to reveal a bug.

actions, as these single-turn perturbations are insufficient to verify the context dependence intrinsic to multi-turn dialogues. In Fig. 1, with the original test case as a test seed, dialogue-level perturbation produces test cases with nuanced context information for the same round input and reaches significantly enlarged dialogue coverage.

In this paper, we propose MORTAR, a metamorphic multi-turn dialogue testing approach, which mitigates the test oracle problem in the assessment of LLM-based dialogue systems. We focus on the multi-turn QA dialogue [17], and use the open-domain multi-turn dialogue dataset [18] to form follow-up test cases to implement multi-turn MT for LLM-based dialogue systems. MORTAR formalise four MRs and proposes five dialogue-level perturbations, using the multi-turn MT framework to generate follow-up test cases and reveal bugs in dialogue systems. The matching of MR is not conducted manually, but can be automated with an equivalent context check process. Such that MORTAR is fully automated, and does not require an LLM as a judge. According to the experimental results on multiple LLM-based dialogue systems, MORTAR achieves higher effectiveness in testing. Particularly, MORTAR reveals 150% more bugs than the most effective single-turn MT baseline [16]. Regarding the quality of bugs, MORTAR achieves better performance in terms of diversity, precision and uniqueness. An ablation study verifies the contribution of MRs and perturbations to the overall effectiveness.

The main contributions of this paper are as follows:

- 1) We formalise the framework of multi-turn metamorphic testing, which serves as the foundation of MORTAR.
- 2) Three basic and two derivative dialogue-level perturbations are introduced to produce follow-up test cases.
- 3) Four novel MRs are proposed and adaptively used with dialogue-level perturbations to reveal bugs in LLM-based dialogue systems. The entire testing process is fully automated.
- 4) MORTAR successfully reveal high-quality bugs in different dialogue systems regarding quantity, diversity, precision and uniqueness. Experiments show that MORTAR outperforms the most effective single-turn metamorphic testing approach in identifying unique bugs.

The rest of this paper is structured as follows: Section II introduces the background and motivation. Section III elaborates on the details of MORTAR. Section IV describes the settings of experiments. Section V presents the experiment results analysis and discussion. Section VI reports threats to validity. Finally, Section VII introduces related works and Section VIII concludes and discusses potential future works.

II. BACKGROUND AND MOTIVATION

This section provides the necessary background to contextualise our work, covering key areas including the current landscape of testing LLM-based dialogue systems, the role of MT in addressing oracle challenges, and the critical distinction between single-turn and multi-turn testing approaches.

A. Testing and evaluation of LLM-based dialogue systems

Dialogue systems are now broadly applied in many domains, especially after the recent rapid development of LLMs [19], [20]. To ensure the quality of natural language generation applications, multiple datasets and evaluation methods emerged to test the dialogue system from many different aspects [5], [21], [22]. The test datasets typically comprise test input utterances and the expected outputs. Given a certain dialogue context and an input utterance, a bug detection is performed to judge if the output of the dialogue system meets the correctness criteria. If not, the detection will be recorded as positive to indicate a bug of the dialogue system. It is the common practice to test using such reference-based scheme [23], namely reference-based testing (RBT). The test case that reveals at least one bug is recognised as an *effective* test case; otherwise it is considered *ineffective*.

The testing of dialogue systems can be broadly categorised into a single-turn or a multi-turn. In single-turn testing, the system is provided with one input utterance at a time, and its response is evaluated against expected behaviour [16]. In contrast, multi-turn testing involves a sequence of inputs and outputs, simulating real conversational flows where the system must maintain context, coherence, and consistency across multiple exchanges [24]. Most existing approaches for testing LLM-based dialogue systems are single-turn.

Single-turn testing faces several well-known challenges. One major issue is the difficulty and high cost associated with acquiring reliable testing datasets [14]. Additionally, there are growing concerns about potential data contamination in existing LLM training datasets [25], raising questions about the continued effectiveness of tests based on such datasets. These challenges are amplified in multi-turn testing. A multi-turn test case is a series of simulated user inputs of multiple utterances with corresponding expectations on output. High-quality multi-turn dialogue test datasets are significantly less available than their single-turn counterparts [26]. A recent survey [7] found that only 2 out of 46 evaluations focus on multi-turn interactions.

To mitigate the oracle problem, prompting LLMs and directly generating test datasets are adopted in an increasing number of studies. Latest multi-turn testing focuses on using LLM to generate test cases and invite LLMs to function as

judges to score the output of LLM-based dialogue systems [10], [11], [27]–[29]. The disturbing point of using LLMs to generate test cases for LLM-based dialogue system testing is that LLMs are trained on massive datasets to converge and act normally, but testing requires test cases to be unfamiliar and diversified. Using LLM to generate test cases and evaluate the outputs faces issues such as lack of diversity of test cases and potential bias in evaluation [12]. These factors lead to the unsatisfactory multi-turn testing of LLM-based dialogue systems. There exists a notable gap between current testing approaches and expectations of LLM-based dialogue systems.

B. Metamorphic testing

RBT relies on dataset annotation as the test oracle to judge the quality of output, and it is not feasible if the test oracle is inaccessible or no-longer effective. MT was first proposed by Chen et al. in 1998 [13] to test numerical programs without reliance on test oracles. Over past development, MT has been widely used to mitigate the oracle problem in software testing, especially for those low-resource testing scenarios with limited availability of testing datasets [30]. The fundamental idea of MT is to formalise MRs and generate follow-up test cases from the *test seeds*, then check if the output of the system with follow-up input violates with the MRs [31]. The violation will be regarded as a system bug.

According to recent researches, MT has been widely used in both testing traditional software systems and machine learning software systems [32], [33]. Test seed is the original test case that can be used to produce follow-up test cases. In testing dialogue systems, MT is promising in exploring more unique bugs with limited testing seeds [14], [16], [34]. However, there is no best practice for applying MRs in multi-turn dialogue testing. Previous testing is generally carried out in a single turn manner where the perturbations applied to the dialogue system are low-level, including character-level, word-level, and sentence-level [35], [36].

Considerable single-turn MT methods are not applicable to multi-turn scenarios. For example, QAQA [15] is one of the latest single-turn MT approaches for dialogue systems. It splits the testing dataset into two parts and uses one as a searching space to find applicable perturbation elements and add to the target question. In multi-turn testing, QAQA is not feasible as: a) many questions in dialogue are too short to accurately calculate semantic of an individual short sentence is unclear, which makes searching for related content infeasible. b) QAQA expects the candidate utterance from searching space to be self-contained and feasible to be injected into the target question, yet considerable questions in multi-turn dialogue are context-sensitive. Thus they cannot be inserted into other dialogues, and the dataset itself can not be used as a reliable searching space.

Another major difference between single-turn and multi-turn MT is that, for single-turn MT, perturbations are generally pre-defined to be either semantic-preserving or semantic-altering, and they will be manually matched with corresponding MR and then be used in MT [14]–[16], [35]. In multi-turn MT, the effect of dialogue-level perturbations might need additional

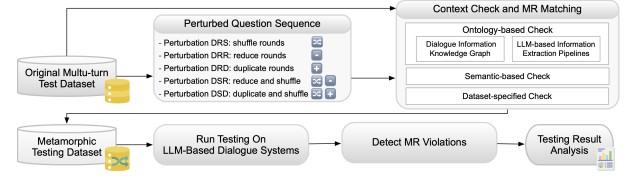


Fig. 2. An overview workflow of MORTAR.

processing to judge the actual effect on the test seeds. It is necessary yet difficult to tell whether a target question in the perturbed test case is given sufficient information from context to be answerable. This brings an additional challenge to multi-turn MT. Dialogue-level perturbation has not been proposed for its intrinsic difficulty in modelling and implementation.

In summary, realising multi-turn MT for dialogue system remains challenging for the following reasons:

- Relatively limited availability of multi-turn test seeds
- Absent of formalised multi-turn MRs
- Lack in dialogue-level perturbations
- Difficulty in test automation

To our knowledge, we are the first to systematically introduce dialogue-level MRs with implementation of effective and automated metamorphic test case generation pipelines and MR violation detection to mitigate the oracle problem in multi-turn dialogue system testing.

III. APPROACH

In this section, we introduce the framework of MORTAR, the formalised MRs and perturbations, and a feasible design of test automation.

A. Overview of MORTAR

As shown in Fig. 2, given the original dialogue dataset, MORTAR first generates the perturbed follow-up test input with three major and two derivative dialogue-level perturbations (Section III-C). Second, the context check verifies if the question is after an equivalent context to the original context. In each round, the context check result will be used to match each question with the proper MR for bug detection (Section III-D). Finally, after running follow-up tests on LLM-based dialogue systems, MORTAR detects violations of MRs in dialogue system outputs. Ideally, if the dialogue systems can pass the RBT with original test cases, they are expected to pass MORTAR as well. In this way, the original test cases are reused as the test seeds, and MORTAR generate metamorphic test cases to further test the dialogue systems.

B. Formalise Multi-turn Metamorphic Testing Framework

In MORTAR, we focus on the open-domain static multi-turn QA testing scenarios where the correct answer to each question is unique under certain context. For each piece of dialogue D , it is composed of n question answer pairs:

$$D = [(q_1, a_1), (q_2, a_2), \dots, (q_n, a_n)] \quad (1)$$

where the orderings of (q_i, a_i) matters, and all questions q_i and answers a_i in D can be regrouped as two lists, Q and A :

$$Q = [q_1, q_2, \dots, q_n], \quad (2)$$

$$A = [a_1, a_2, \dots, a_n]. \quad (3)$$

We use subscripts on lists to denote the initial subsequence:

$$D_i := [(q_1, a_1), \dots, (q_i, a_i)], \quad (4)$$

$$Q_i := [q_1, \dots, q_i], \quad (5)$$

$$A_i := [a_1, \dots, a_i], \quad (6)$$

where $i \in \{1, \dots, n\}$, D_i is the first i elements of D , similarly for Q_i and A_i , such that D_{i-1} can be regarded as context information when the dialogue system receives q_i , and the context of q_1 is empty, i.e. $D_0 = []$. An ideal dialogue system, $IDS(\cdot)$, exhibits the following behaviour:

$$IDS(D_{i-1}, q_i) \rightarrow a_i. \quad (7)$$

Equation (7) represents questions that require contextual information, which is common in multi-turn dialogue. The talker usually refer to information or requirements given in context with pronouns or other types of ellipsis [37] for simplicity and convenience. However, if the contextual dependence is disrupted by perturbations and the critical information is missing in context, the target question shall not be answered with the original answer. In other cases, the target question remains context equivalent if: a) the redundant or unrelated rounds to the target question are removed by perturbations; or b) the target question can be answered independently without reliance on the context, e.g., the first question in original dialogue (q_1), which is usually information-complete and can be answered under empty context.

To take a close look at the context dependency in Equation (7), the context of q_i in input, D_{i-1} , is composed of the previous context D_{i-2} , the question q_{i-1} , and the answer a_{i-1} . The answer a_{i-1} in the previous round is further produced by earlier round:

$$IDS(D_{i-2}, q_{i-1}) \rightarrow a_{i-1}. \quad (8)$$

Such that the information in a_{i-1} has been implied by question q_{i-1} and its context D_{i-2} . Equation (7) can be rewritten as:

$$IDS([D_{i-2}, q_{i-1}], q_i) \rightarrow a_i. \quad (9)$$

Similarly, since all previous answers are dependent on the sequence of questions in the corresponding context, Equation (9) can be expressed as:

$$IDS([q_1, \dots, q_{i-1}], q_i) \rightarrow a_i. \quad (10)$$

Or, after the definition of Q_{i-1} , as follows:

$$IDS(Q_{i-1}, q_i) \rightarrow a_i. \quad (11)$$

It is intuitive that the answer can be regarded as implied in the question sequence of context.

Conducting RBT on a dialogue system $DS(\cdot)$ with the original test case, is operated as feeding the dialogue system with question sequence Q and then verifying the quality of

each output. Given the context question sequence Q_{i-1} and the target question q_i , the dialogue system generates responses o_i :

$$DS(Q_{i-1}, q_i) \rightarrow o_i. \quad (12)$$

It is expected that the dialogue system under test generate similar answers to the IDS when fed with the same inputs:

$$\forall i \in \{1, \dots, n\} (\Delta(o_i, a_i) < \epsilon) \quad (13)$$

that is:

$$\forall i \in \{1, \dots, n\} (\Delta(DS(Q_{i-1}, q_i), IDS(Q_{i-1}, q_i)) < \epsilon) \quad (14)$$

where $\Delta(\cdot)$ measures the difference between two natural language sentences. In RBT, the difference is expected to be smaller than a threshold ϵ . Otherwise, a bug will be reported under the current test case. One dialogue test case might be capable of revealing multiple bugs in different rounds, which will be recognised as different bugs revealed by this particular test case.

Define the follow-up test case as Q' :

$$Q' = [q'_1, q'_2, \dots, q'_{n'}] \quad (15)$$

where $n' \geq 2$, n' and n may not be equal. Practically, Q' can be constructed from Q through perturbation operations, e.g., element permutation, element deletion, element addition, and compositions of these operations, that is:

$$\forall i \in \{1, \dots, n'\} \exists j \in \{1, \dots, n\} (q'_i = q_j \text{ and } q_j \text{ appears in } Q) \quad (16)$$

After perturbation, each q'_i in Q' may have different context when compared to their identical question q_j in Q . If q'_i and q_j deliver the same answer, then we regard Q'_{i-1} and Q_{j-1} as equivalent context and denote them as $Q'_{i-1} \equiv Q_{j-1}$. With the equivalent context, it is expected that the output from dialogue system, o'_i , should be similarly close to the answer of IDS a_j when compared with o_j . That is, we have:

$$\forall i \in \{1, \dots, n'\} (\Delta(o'_i, a_j) \approx \Delta(o_j, a_j)) \text{ iff } (Q'_{i-1} \equiv Q_{j-1}) \quad (17)$$

In practice, if a_j is inaccessible and verification of $Q'_{i-1} \equiv Q_{j-1}$ is feasible, we have the following:

$$\forall i \in \{1, \dots, n'\} (\Delta(o'_i, o_j) \leq \epsilon) \text{ iff } (Q'_{i-1} \equiv Q_{j-1}) \quad (18)$$

where ϵ is the similarity thresholds. If the MR is violated in follow-up testing, a bug will be revealed using target question q_j and test seed D .

C. Dialogue-level Perturbations in MORTAR

Unlike previous perturbations that operate at the single-turn [15], [16], [35], MORTAR employs dialogue-level perturbations. Existing perturbations in single-turn MT can only affect the target question utterance. In contrast, dialogue-level operations are more substantial, as they can alter the conversational context, thereby influencing the answerability of the target question in perturbed test cases. This shift enables the evaluation of language models in more complex, multi-turn settings. Although these modifications are broader in scope, they are still considered perturbations - systematic variations introduced to assess the quality of dialogue systems.

Given the original question sequence, $Q = [q_1, q_2, \dots, q_n]$, the dialogue-level perturbation r is implemented with $P^r(\cdot)$ that produces a new sequence with n^r questions:

$$Q^r = P^r(Q) \quad (19)$$

$$Q^r = [q_1^r, q_2^r, \dots, q_{n^r}^r]. \quad (20)$$

Let δ^r be the change of number of rounds after applying r :

$$\delta^r = |n^r - n|. \quad (21)$$

Each question $q_i^r, i \in \{1, \dots, n^r\}$ in Q^r can be found in Q , but the round that q_i^r appears in Q^r might be different from the round in which the identical question q_j appears in Q , i.e. i might not equal to j , thus q_i^r might have an inequivalent context when compared to q_j . Such situations will need additional judgment to tell if the condition in Equation (17) is met. This is one of the main differences of dialogue-level perturbations, whose effect is variable for each utterance in the test seed. However, this characteristic enables them to generate rarely-seen test cases for dialogue systems when compared with single-turn perturbations, and requires the dialogue system under test to adapt to the actual dialogue and output wisely, indicating higher potential effectiveness of testing.

In MORTAR, three fundamental and two derivative dialogue-level perturbations are proposed to produce new question sequences.

- **Perturbation DRS:** Dialogue round shuffle:

$$Q^{\text{DRS}} = P^{\text{DRS}}(Q) = [q_{i'} | i' = \varphi_{\text{DRS}}(i), i = 1, \dots, n] \quad (22)$$

where $\varphi_{\text{DRS}}(i)$ is a bijective mapping from the original round index to the shuffled round index. All original questions are reordered and form the perturbed question sequence. Perturbation DRS can be regarded as element permutation of Q .

- **Perturbation DRR:** Dialogue round reduction:

$$Q^{\text{DRR}} = P^{\text{DRR}}(Q) = [q_i | \mathbb{I}(i) = 1, i = 1, \dots, n] \quad (23)$$

where $\mathbb{I} : \{1, \dots, n\} \rightarrow \{0, 1\}$ is a randomly sampled mask function, $\sum_{i=1}^n \mathbb{I}(i) = n - \delta^{\text{DRR}}$, and each element in Q^{DRR} is from Q . That is, δ^{DRR} questions are randomly removed from the original sequence, and the rest form the perturbed question sequence. Perturbation DRR can be regarded as element deletion of Q .

- **Perturbation DRD:** Dialogue round duplication:

$$Q^{\text{DRD}} = P^{\text{DRD}}(Q) = \text{Insert}(Q, X), \quad (24)$$

where X is a random subset of Q with δ^{DRD} elements, the function $\text{Insert}(\cdot)$ iteratively inserts each element from X into a random position between two elements within the target question sequence or the head or tail position, finally forms the perturbed question sequence Q^{DRD} with $n + \delta^{\text{DRD}}$ elements. Inside it, all original questions from Q are preserved as the original order with potential intervals, and each selected question in X appears one additional time at random positions. Perturbation DRD can be regarded as element addition of Q .

Perturbation DRS, DRR and DRD are three fundamental dialogue-level perturbations. They simulate complicated real-world usage of dialogue systems in diverse situations.

Additionally, two derivative perturbations are employed. The first is Perturbation DSR: dialogue round shuffle and reduction, which combines DRS and DRR. The second is Perturbation DSD: dialogue round shuffle and duplication, which combines DRS and DRD.

- **Perturbation DSR:** Dialogue round shuffle and reduction is defined as

$$Q^{\text{DSR}} = P^{\text{DSR}}(Q) = P^{\text{DRS}}(P^{\text{DRR}}(Q)). \quad (25)$$

Perturbation DSR is a combined operation of DRS and DRR. It deletes δ^{DRR} randomly chosen elements from the original question sequence and shuffles the rest to form a perturbed question sequence. It simulates more complicated information-oriented real-world requests in long contexts, and tests the performance of dialogue systems.

- **Perturbation DSD:** Dialogue round shuffle and duplication is defined as

$$Q^{\text{DSD}} = P^{\text{DSD}}(Q) = P^{\text{DRS}}(P^{\text{DRD}}(Q)). \quad (26)$$

Perturbation DSD is a combined operation of DRS and DRD, which duplicates δ^{DRD} randomly chosen questions, inserts the chosen questions into random positions and finally shuffles the new sequence to form a perturbed question sequence. It requires dialogue systems to tell if sufficient information is given in context to answer the target question, and perform different behaviours. This perturbation further extends the test coverage of dialogue systems with rarely seen situations, and is likely to contribute to the uniqueness of revealed bugs.

Testing the dialogue system with a perturbed question sequence brings new challenges to the system's dialogue capability of context understanding and suppressing mistakes that were made in previous rounds. The effect of dialogue-level perturbations is variable. In some cases, the expectation of answers in the perturbed test cases may differ from answers in other perturbations and the original answer. It is not proper to use a static MR, since different conditions may require different MRs to be adopted upon violation detection. To handle this issue and implement metamorphic testing, it is necessary to dynamically use the appropriate MR according to the context of each question in the perturbed test case.

D. MRs in MORTAR

For one specific question in the original question sequence, the context can be changed using dialogue-level perturbations. If the context remains equivalent to the original context, the dialogue system is expected to respond with a semantically similar answer, regardless of whether the question is asked in the RBT or MT. Otherwise, for the same question with different informative contexts, the dialogue system is expected to respond with semantically different answers.

The equivalent context check function $EC(\cdot)$ returns true if, Q_{i-1}^r , the context of target question $q_i^r, i \in \{1, \dots, n^r\}$ is equivalent to Q_{j-1} , which is context of q_j in Q :

$$EC(Q_{i-1}^r, q_i^r) = \begin{cases} \text{True}, & \text{if } Q_{i-1}^r \equiv Q_{j-1} \\ \text{False}, & \text{Otherwise} \end{cases} \quad (27)$$

Given the test seed Q and a set of perturbation operations R , each r in R is one of the independently executed perturbation among DRS, DRR, DRD, DSR and DSD, meaning the same perturbation in two execution yield different test cases. For the original test input from Q_j and perturbed test input from Q_i^r , the dialogue system $DS(\cdot)$ respectively generates responses as follows:

$$DS(Q_{j-1}, q_j) \rightarrow o_j, \quad (28)$$

$$DS(Q_{i-1}^r, q_i^r) \rightarrow o_i^r, \quad (29)$$

where $i \in 1, \dots, n^r, j \in 1, \dots, n, q_i^r = q_j, q_i^r$ is the target question in perturbed test case, q_j is the identical question in original test case, o_i^r and o_j are outputs in MT and RBT respectively, Q_{i-1}^r and Q_{j-1} are context in MT and RBT respectively.

Using equivalent context check function, MR1 and MR2 are defined as follows:

- **MR 1: Context-Preserving MR** If a perturbed context Q_{i-1}^r is equivalent to Q_{j-1} in terms of supplementary information to the target question:

$$EC(Q_{i-1}^r, q_i^r) = EC(Q_{j-1}, q_j) = \text{True}, \quad (30)$$

the dialogue system is expected to produce an answer semantically similar to the original expected response:

$$\text{Expect: } \Delta(o_i^r, a_j) < \epsilon. \quad (31)$$

- **MR 2: Context-Altering MR** If perturbed context Q_{i-1}^r of q_i^r lacks critical information to clarify q_i^r :

$$EC(Q_{i-1}^r, q_i^r) \neq EC(Q_{j-1}, q_j), \quad (32)$$

the dialogue system is expected to produce an answer semantically different from the original expected response:

$$\text{Expect: } \Delta(o_i^r, a_j) \geq \epsilon. \quad (33)$$

All perturbations can be fitted with MR1 or MR2. Given the context check results, it can be determined which MR and perturbation shall be matched for bug detection. In addition to individual MRs, we further propose clustering-based MRs that expect dialogue systems to adapt to and ensure consistent behaviour among different versions of context-changed questions.

For an original target question q_j from the original dataset, assuming there are multiple perturbed question sequences where the identical target question appears at different rounds with different contexts. We name the same question under different contexts as different *versions* of the original question q_j . After executing different perturbations in R , all different versions of q_j form a group q_j^R , and the all outputs of the dialogue system under test form a group o_j^R :

$$q_j^R = \{q_i^r | q_i^r = q_j, r \in R\} \quad (34)$$

$$o_j^R = \{o_i^r | o_i^r = DS(Q_{i-1}^r, q_i^r), q_i^r = q_j, r \in R\} \quad (35)$$

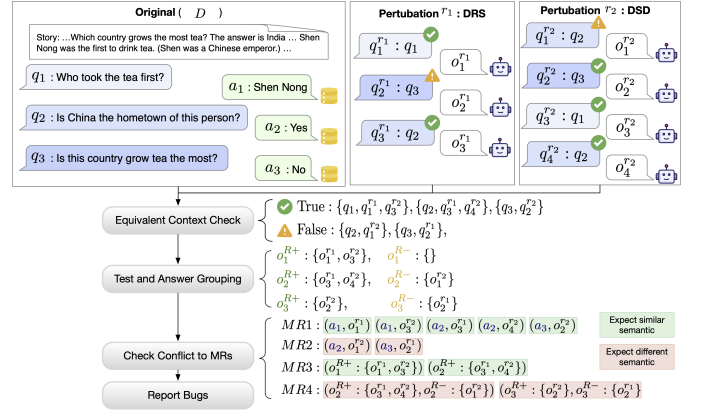


Fig. 3. Perturbation and MR violation detection.

The context of each q_i^r in corresponding test case may be equivalent or inequivalent to the original context of q_j in test seed. We split the outputs o_j^R into the output group of context consist perturbations of the original question, o_j^{R+} , and the output group of context altered perturbations, o_j^{R-} :

$$o_j^{R+} = \{o_i^r | EC(Q_{i-1}^r, q_i^r) = \text{True}, q_i^r \in q_j^R\}, \quad (36)$$

$$o_j^{R-} = \{o_i^r | EC(Q_{i-1}^r, q_i^r) = \text{False}, q_i^r \in q_j^R\}. \quad (37)$$

- **MR 3: Inner-Group Consistency MR** Within group o_j^{R+} , different versions of q_j have equivalent context to the original, the dialogue system should not produce highly divergent answers, meaning the maximum difference within the group should be below threshold ϵ :

$$\text{Expect: } \max_{o_1, o_2 \in o_j^{R+}} \Delta(o_1, o_2) < \epsilon. \quad (38)$$

- **MR4: Inter-Group Divergence MR** For two versions of q_j from the two groups, the system's outputs must be different and greater than ϵ :

$$\text{Expect: } \min_{o_1 \in o_j^{R+}, o_2 \in o_j^{R-}} \Delta(o_1, o_2) \geq \epsilon. \quad (39)$$

As is shown in Fig. 3, given the original multi-turn test case (D) as test seed, two perturbations r_1 (DRS) and r_2 (DSD) are adopted. There, the questions in the original round are given different contexts in perturbed test cases, resulting in different $EC(\cdot)$ results. After equivalent context checks on each question, the outputs from different perturbed inputs are then grouped by target question and $EC(\cdot)$ results. With the above processing, MRs will then be matched for violation detection. For MR1, take perturbation r_1 for example, the original question in the second round q_2 appeared in the third round of r_1 perturbed dataset as $q_3^{r_1}$. The output of this round is $o_3^{r_1}$. We expect the semantics of a_2 and $o_3^{r_1}$ to be similar, otherwise a bug is found using the original test case D as test seed with perturbation r_1 and metamorphic relation MR1. On the contrary, take perturbation r_2 for example, the first question $q_1^{r_2}$ in the generated dataset is unanswerable, if the semantic of a_2 and the output $o_1^{r_2}$ are not sufficiently different, a bug is revealed using test seed D , perturbation r_2 and MR2.

To our knowledge, the above formalisation is the first attempt to model the multi-turn MT for dialogue systems. It is

worth noting that MORTAR’s process is different from single-turn metamorphic testing methods. Existing methods predefine the perturbation and select the proper MR in testing. This process is inevitably semi-automated as manual matching of MR for perturbation is necessary. In multi-turn MT, the effects of dialogue-level perturbations are determined later with the equivalent context check process. This mechanism allows MORTAR to enhance the diversity of test cases and maximise the usage of test seeds. Given that $EC(\cdot)$ is automated, the metamorphic testing can be fully automated.

E. Equivalent Context Check and Its Automation

The equivalent context check in Equation 27 returns whether the context of a question in a perturbed test case is equivalent to its context in the original test case. We observe that no plug-and-play tool is primarily designed for this intent. As a result, we craft a feasible equivalent context check toolset from the perspective of context information. The proposed equivalent context check is based on the assumption that, if the context of a perturbed test input fails to provide sufficient information, the context will be regarded as inequivalent to the original context which is information complete. Such an assumption is expected to hold strongly for MR2 and MR4, but might exist limitations for MR1 and MR3, as some inequivalent contexts will be mislabelled as equivalent. Special attention is paid to the quality of revealed bugs of MORTAR in experiments to show the feasibility of the adopted equivalent context check process. The intuition is to utilise existing natural language processing pipelines and tools to automatically extract the critical information in perturbed test case, and verify if the context information of each question is as complete as the original context.

1) *Context Check Toolset in MORTAR*: The equivalent context check in MORTAR comprises the semantic-based check, the ontology-based check, and a dataset-specified check. These three checks validate if the question itself, its context, or other reference material from the dataset provides sufficient information to realise equivalence, such that the proper metamorphic relation can be matched in bug detection. If one of the checks indicated positive, the corresponding question is regarded as having equivalent informative context to the original. In these checks, the original answers will be used to assist the analysis.

a) Semantic-based check: Extracting information in context can be regarded as the problem of anaphora resolution. Anaphora is defined as the phenomenon of pointing back a previously mentioned item in the text. [38], [39]. If a question does not require additional information from the context, it will be regarded as self-resolved. Questions with fewer than three words are excluded from this check as they are too short to carry sufficient information to become independently answerable. Pronouns are typical anaphora in sentences [39], which link critical information about a person or an object mentioned before. Semantic-based resolution checks aim to use end-to-end models to extract pronoun-resolution information reliance within the question sentence itself. If the target question passes this check, the question can be regarded as independently answerable regardless of the context, thus the context will be regarded as equivalent to the original context.

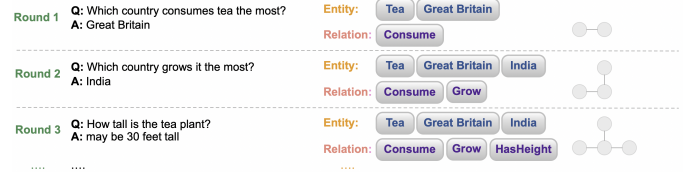


Fig. 4. In ontology-based check, as the dialogue proceeds, information in context is accumulated as a reference for following questions.

b) Ontology-based check: We use a knowledge graph-based dialogue information model to realise the ontology-based check. We regard all entities and relations involved in the whole dialogue’s utterance D to form a whole knowledge graph $G(V, E)$. Before the occurrence of the first round, the context information graph $G_0(V_0, E_0)$ is empty. As shown in Fig. 4, as the dialogue proceeds, entities and relations in a round are extracted and added to the context information graph, such that the next round’s context information graph will contain information extracted from the previous rounds as reference. Here, answers from original test cases are used to clarify what information is implied in questions.

For a question in round j of the original dialogue, the context information is $G_j(V_j, E_j)$. There might exist unnecessary entities and relations with q_j . The necessary part is noted as $G_j^r(V_j^r, E_j^r)$. If $G_j^r = (\emptyset, \emptyset)$, the question contains all information and can be answered independently, and this check will indicate a pass. In a perturbed test case, the entities and relations mentioned in the context of round i form $G_i^r = \bigcup_{n=1}^{i-1} G_n^r$. If q_j and q_i^r are the same question, and $G_j^r \subseteq G_i^r$, the context of q_i^r can be regarded as similarly informative as the original context, then this check will indicate a pass. Besides, information explicitly and implicitly given by q^j will be added to the next question’s context G_{i+1}^r . Otherwise, the context will be regarded as lacking critical information, and only the explicitly mentioned information in the question itself will be added to the context of the next question.

c) Dataset-specified check: For reading comprehension datasets, an additional semantic-based check will be added. Such that even if some information has never been mentioned in dialogue context, as long as it is used in other supplementary material, e.g. the reference story provided, this ellipsis information will be regarded as reachable, namely story-resolved. The dataset-specified check is realised similarly to the Semantic-based check.

The above three checks compose the equivalent context check in MORTAR. If any check indicates pass, the context of the perturbed question will be regarded as equivalent to the original context. Practically, violations of MR1 will be detected to reveal bugs. Otherwise, violations of MR2 will be detected. Verification of MR3 and MR4 will be conducted upon the grouping of outputs using context equivalence.

2) *The Automation of Equivalent Context Check*: The semantic-based check and dataset-specified check can be automated with existing end-to-end models. We propose a feasible implementation of ontology-based check using LLM-based information extraction pipelines. Inspired by GraphRAG and EDC approach for knowledge graph construction [40], [41],

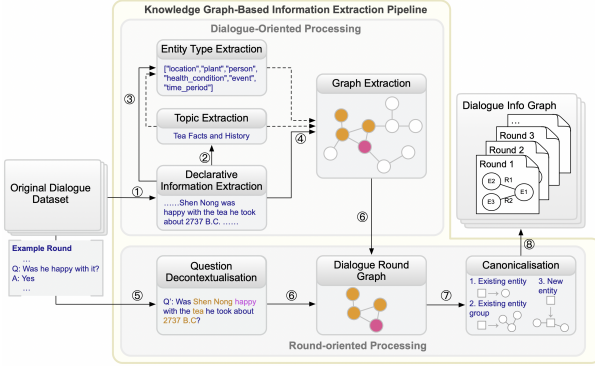


Fig. 5. Information Extraction Procedure with LLM pipelines

a series of well-designed single-turn information extraction (IE) prompt templates are used to process dialogue data and generate structured data in JSON format. With these prompt templates, we implemented information extraction pipelines with the following functions:

- Declarative information extraction: turn each round into a declarative sentence.
- Topic Extraction: extract the topic of a dialogue.
- Entity Type Extraction: extract a list of entity types in a dialogue.
- Graph Extraction: extract all entities from entity types, and relations between entities, each entity and relation will also be given a description.
- Question Decontextualization: turn each question into an independently answerable sentence.
- Dialogue Round Graph: choose entities and relations involved in each question and answer from provided entities and relations.
- Canonicalisation: determine whether target entity is a group of existing entities. If no matches, return the entity and most possible entity type.

As is shown in Fig. 5, the declarative information is first extracted, and then it is used to extract the dialogue topic and entity types. With these information, the whole graph with all entities and relations is extracted. Using round’s decontextualised sentences, we extract a subgraph of each round from the whole graph. Finally, canonicalisation will handle the unseen entity and judge if it is an alias of an existing entity or a group of existing entities, and will update the whole graph if it is an omitted entity.

In this way, all three components in context check are automated, and MORTAR is a fully automated testing method without human interference.

IV. DESIGN OF EXPERIMENTS

In this section, we introduce the research questions, setting of experiments, baseline methods, the dataset, dialogue systems under test, and relevant implementation details.

A. Research Questions

RQ 1: How does MORTAR perform in detecting bugs in LLM-based dialogue systems? We answer this research ques-

tion by comparing the effectiveness, efficiency, and robustness of MORTAR with the baseline method.

RQ 2: What is the quality of the bugs revealed by MORTAR? In this RQ, we analyse the characteristics and quality of detected bugs in terms of diversity, precision and uniqueness.

RQ 3: How effective are the different metamorphic relations and perturbations of MORTAR? In this RQ, we conduct an ablation study and investigate the contribution of the five perturbations and four MRs to the overall performance.

B. Experiment Environment

The local machine is equipped with 32-core processors, 64GB memory and an RTX 3090 GPU. The system runs Ubuntu 24.04 LTS and Python 3.9. Groq² is an LLM inference cloud service provider. Their API provides rapid LLM inference of models with sizes up to 70 billion parameters. In MORTAR, the testing for open source LLM-based dialogue system is conducted locally, the models are obtained through Hugging Face³. Using the fine-tuned model weights, we implemented a series of dialogue systems powered by conversational LLMs with different sources and model sizes. The LLM-based information extraction pipelines introduced in Section III-E2 are implemented with Groq inference API since the computing capacity of our local machine is insufficient for this task. To ensure deterministic behaviour in extraction and analysis, we set the temperature of LLM inference in all steps to zero to ensures relatively stable outputs.

C. Data Preparation

Regarding the test dataset, we have the following expectations: *a)* focusing on fundamental capabilities of testing dialogue systems; *b)* have only two human participants with one posing questions and another answering the questions; *c)* questions might have reliance on dialogue history; *d)* questions can be independently answered when given sufficient information so that the question can be decontextualised. CoQA [18] fundamentally meets all the above requirements. CoQA is a multi-turn reading comprehension dataset with topic in multiple domains. Each record is composed of a story and a multi-turn dialogue record. The multi-turn dialogue is produced by a human question-asker and a human answerer. It comprises a training set and a development set. We adopt the development set as test seeds. There are a total of 500 dialogues with 7983 questions. There are minor questions that are designed to be unanswerable and expect the answer of “Unknown”. We exclude them in MR violation detection in MORTAR to reduce the complexity of the process.

D. Baseline of MT Approach

METAL [16] is one of the latest and most effective MT frameworks for single-turn LLM testing without reliance on LLM judges. We choose the four most effective single-turn MRs in METAL: the model’s outputs on the original and the perturbed input should not differ, the perturbations are

²<https://groq.com/>

³<https://huggingface.co/models>

synonym-replacement, add random word, introduce-typos, and convert-to-leet-format. The number of MRs in baseline is the same as the number of MRs in MORTAR. We independently adopt each of the single-turn perturbations on the dataset and form the follow-up test cases, and verify if the output of the perturbed round violates the MRs. The perturbed questions with significant semantic change will be revoked to satisfy the condition of MRs. Besides, all originally unanswerable questions will be regarded as answerable questions with the answer expectation of “Unknown”.

E. Test Objects

In terms of SUTs, we select six popular open-source conversational LLMs and prompt each of them to function as a dialogue system (DS). In the prompts, the story is first provided as a topic material, then the LLMs are required to answer questions with *concise and short* answer to reduce unrelated information, and they are required to answer “Unknown” if they do not know the answer. The selected LLM for each dialogue system are as follows:

- DS1: Qwen2-0.5B-Instruct [42]
- DS2: Qwen2-1.5B-Instruct [42]
- DS3: Qwen2-7B-Instruct [42]
- DS4: Mistral-7B-Instruct-v0.3 [43]
- DS5: Meta-Llama-3-8B-Instruct [44]
- DS6: Gemma-2-9b-it [45]

The models used are all instruction fine-tuned and their sizes range from 0.5 to 9 billion parameters. These dialogue systems can be regrouped into two sets: the first group is composed of DS1, DS2, and DS3. They are the same sourced models with different parameter sizes. The second group is composed of DS3, DS4, DS5 and DS6. They are all based on popular and recently published language models with similar model sizes. When testing these dialogue systems, we simulate a user having multi-turn dialogues with the dialogue system. We feed the dialogue system with the question sequence in each meta-morphic test case along with the dialogue history in previous rounds, then collect the generated answer to analyse response quality. The *positive detection* of MR violation indicates a bug in the dialogue system, and further system development or patching is necessary for performance improvements.

F. The Result of RBT

When conducting RBT all dialogue systems under test with original test cases, all dialogue systems are exposed to the same test set of 7,983 questions from 500 original dialogue test cases, according to the result in Table I. The results show substantial variance in performance: the number of detected bugs ranges from 1,999 (DS6) to 6,654 (DS1), and the positive rate varies from 25.0%(DS6) to 83.3%(DS1). This result indicates that different dialogue systems exhibit significantly different behaviours and robustness levels.

These results illustrate the necessity of further testing for dialogue systems. Different dialogue systems exhibit differently using the same test dataset. DS1 exhibits a positive rate of 83.3%. In follow-up testing, it is expected to explore how these

TABLE I
TEST RESULT OF RBT USING ORIGINAL TEST CASES.

	DS1	DS2	DS3	DS4	DS5	DS6
Test cases	500	500	500	500	500	500
Total questions	7,983	7,983	7,983	7,983	7,983	7,983
Number of bugs	6,654	5,582	5,028	5,014	3,042	1,999
Positive rate	83.3%	69.9%	63.0%	62.8%	38.1%	25.0%
Effective test cases	500	500	495	500	485	470

TABLE II
METRICS USED TO EVALUATE THE PERFORMANCE OF TESTING METHODS.
DIRECTION (DIR.): ↑ HIGHER-IS-BETTER, ↓ LOWER-IS-BETTER.

Metrics	Explanation	Dir.
RETC	Ratio of effective test cases (reveals at least one bug) to all test cases.	↑
BPTC	Bug per test case, the number of bugs detected per test case.	↑
NBugs	Number of positive detections.	↑
Rate ⁺	Positive rate in testing, number of positive detections divided by the number of total detections.	↑
CV	Coefficient of Variation, the measurement the diversity.	↓
PPD	Precision of positive detection, measured with manual check.	↑
L1-Bugs	L1-Bugs are replicated bugs in RBT but with different triggering contexts in MT.	↑
L2-Bugs	L2-Bugs are discovered in MT using correctly answered questions of effective test cases of RBT.	↑
L3-Bugs	L3-Bugs are discovered in MT using ineffective test cases of RBT.	↑

bugs would appear in real-world usage with other triggering conditions. DS6 exhibits a 25% positive rate, indicating the test dataset is significantly less effective when compared to the testing of DS1. An effective test approach is expected to reuse the original test dataset and reveal more new bugs of DS6. To comprehensively assess the performance of each DS, we implement MT to detect more bugs. Besides, the bugs detected by MT but are not detected by RBT will be categorised into different types and credit the MT approach for varied effectiveness.

G. Criteria for multi-turn testing

When analysing the test performance of a multi-turn testing approach, we use a series of multi-turn testing specific metrics to quantitatively describe the performance of a testing method. They are adapted from existing single-turn testing metrics, and are introduced in Table II. The ratio of effective test case (RETC) and bugs per test case (BPTC) are used to measure overall effectiveness. RETC represents the percentage of test cases that detect at least one bug, calculated as:

$$\text{RETC} = \frac{\text{Number of effective test cases}}{\text{Total test cases}} \times 100\%.$$

BPTC is the number of bugs detected per test case, given by:

$$\text{BPTC} = \frac{\text{Total number of detected bugs}}{\text{Total number of test seeds}}.$$

Additionally, Rate⁺ is used to evaluate the bug detection efficiency, exposing more bugs with fewer detection attempts will result in a high Rate⁺.

To delve into the type of revealed bugs, we categorise bugs based on how they are exposed when compared with RBT. An **L1-Bug** is revealed in the condition that, the question is also incorrectly answered in RBT, and it is replicated under a different context in MT to show different trigger conditions of an already revealed bug. An **L2-Bug** is a bug revealed in the condition that, in an effective test case (revealed at least one bug in RBT), a question is correctly answered by dialogue system, while it is mistakenly answered in MT with a different context. An L2-Bug can be regarded as a successful reuse of a test seed from round level. An **L3-Bug** is a bug revealed in the condition that, in an ineffective test case where all questions are correctly answered by DS in RBT, a question is mistakenly answered by the DS in MT with a different context. The L3-Bug can be regarded as a successful reuse from the test seed level, thus it is more distinctive than other types.

Regarding the uniqueness of bugs, a unique L1-Bug detected by an MT method is that, this bug is detected in RBT, while other MT methods failed to provide different conditions to replicate it. Similarly, for unique L2-Bugs and L3-Bugs detected by an MT method, using the ineffective rounds or test cases in RBT, an MT method successfully reveals bugs while other MT methods do not, such that the effective MT method is credited for this uniqueness. The importance of these bugs is deemed to increase with their levels. This classification allows us to assess the testing methods with more fine-grained metrics. The target of an effective MT approach for dialogue systems is to realise more *true positive detections* and reveal more unique L1-Bugs, L2-Bugs, and L3-Bugs using limited test seeds.

Regarding the diversity of bugs, we adopt the coefficient of variation (CV) to measure the diversity, calculated as:

$$CV = \frac{\sigma}{\mu},$$

where σ and μ are the standard deviation and mean of the number of three types of bugs. Higher values of CV indicate greater relative dispersion among the three bug types.

In addition, we use precision of positive detection (PPD) to measure the ratio of true positive detections in manual check, calculated as:

$$PPD = \frac{\text{Number of true positive detections}}{\text{Number of sampled positive detections}} \times 100\%.$$

When comparing performance between multi-turn testing and single-turn testing, greater quantity (NBugs), higher precision (PPD), more uniqueness and diversity of detected bugs indicate better performance of the testing method.

H. Experiment Settings

The semantic similarity threshold ϵ is set to be 0.6, which is the same setting with the baseline. The semantic similarity is calculated with the cosine similarity of two sentences' embedding. We use the commonly used model, sentence-transformers/all-MiniLM-L6-v2⁴ as the embedding model. Besides, Spacy Coreference⁵ pipeline is used to analyse the

TABLE III
SUMMARY OF FOLLOW-UP TEST CASE GENERATION IN METAL AND MORTAR. THE RELATIVE DIFFERENCE IS CALCULATED BY:
(MORTAR - METAL)/METAL $\times$ 100%

	METAL	MORTAR	Difference
Test seeds	500	403	-19.4%
Total test cases	2,000	2,015	+0.8%
Total questions	31,932	30,869	-0.2%
Total detections	14,665	36,908	+151.7%

anaphora resolution. In Perturbation DRR and DSR, 30% of original rounds are randomly chosen to be reduced. In Perturbation DRD and DSD, 20% of original rounds are randomly chosen to be duplicated.

V. RESULTS

In this section, we present the empirical results to answer RQs presented in Section IV-A, and discuss the information extraction, false positive detections, and the performance evaluation of different dialogue systems.

A. RQ1. Effectiveness of MORTAR

RQ1 evaluates the overall performance of MORTAR in revealing dialogue system bugs. Before testing, we generate follow-up test cases. The generation is summarised in Table III. For the baseline method METAL, all original test cases are used as test seeds. For MORTAR, the information extraction process, as depicted in Figure 5, is first executed on all test seeds. As the adopted LLM in IE pipelines exhibits occasional unformatted output or failure of recognising and aligning some entities, 403 out of 500 dialogues are successfully processed and adopted as test seeds in MORTAR. We regard the success rate over 80% as acceptable.

Both METAL and MORTAR generate similar amounts of follow-up test cases. However, the number of feasible MR violation detection varies. Restricted by the mechanism of single-turn MT, METAL is only capable of detecting an upper bound of 14,665 bugs using the generated follow-up test cases. Benefits from dialogue-level perturbations and the automated equivalent context check, each question in MORTAR test cases might participate in detecting multiple different MR violations. As a result, the number of detections of MORTAR exceeds the number of questions by 20%, reaching 36,908.

After testing the six dialogue systems with METAL and MORTAR, the results are presented in Table V-A. We report the overall test performance of MORTAR based on the metrics presented in Section IV-G and summarised in Table II. NBugs is the number of positive detections, measuring the general effectiveness when testing different dialogue system; RETC is the ratio of effective test cases; BPTC is the detected bugs per test case and measures the efficiency of testing; Rate⁺ measures the rate of positive detection against all detections. A higher value for these metrics indicates better test performance.

⁴<https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

⁵<https://spacy.io/universe/project/coreferee>

DS	METAL				MORTAR			
	NBugs	RETC	BPTC	Rate ⁺	NBugs	RETC	BPTC	Rate ⁺
DS1	12,652	100.0%	6.326	86.3%	31,942	100.0%	15,852	86.5%
DS2	10,710	100.0%	5.355	73.0%	27,799	100.0%	13,796	75.3%
DS3	9,754	100.0%	4.877	66.5%	24,634	100.0%	12,225	66.7%
DS4	9,876	100.0%	4.938	67.3%	22,919	100.0%	11,374	62.1%
DS5	6,631	99.8%	3.322	45.2%	17,264	100.0%	8,568	46.8%
DS6	4,199	98.2%	2.138	28.6%	12,805	99.8%	6,371	34.7%
Mean	8,970	99.7%	4.493	61.2%	22,893	100.0%	11,364	62.0%

TABLE IV

THE NUMBER OF DIFFERENT TYPES OF REVEALED BUGS USING MORTAR AND METAL. THE BEST VALUES ACROSS THE DIFFERENT APPROACHES FOR EACH SYSTEM ARE IN BOLD.

DS	METAL				MORTAR			
	L1-Bugs	L2-Bugs	L3-Bugs	CV	L1-Bugs	L2-Bugs	L3-Bugs	CV
DS1	10,903	1,749	0	1.023	27,723	4,219	0	1.041
DS2	8,103	2,607	0	0.726	21,504	6,295	0	0.774
DS3	7,723	2,000	31	0.832	19,083	5,399	152	0.790
DS4	7,463	2,413	0	0.723	18,716	4,203	0	0.896
DS5	4,052	2,490	89	0.338	10,355	6,664	245	0.307
DS6	2,555	1,573	71	0.336	6,819	5,756	230	0.120
Mean	6,799	2,138	31	0.663	17,366	5,422	104	0.654

1) *Effectiveness*: MORTAR and METAL both reveal a significant number of bugs (NBugs). Both methods reach near 100% RETC, indicating almost all follow-up test cases are effective. In terms of the number of detected bugs, MORTAR detect over 150% more bugs than the baseline, indicating a higher effectiveness of MORTAR in testing.

2) *Efficiency*: The average Rate⁺ of MORTAR is 1.3% higher than METAL, and the average BPTC of MORTAR is also 153% higher than METAL. It means MORTAR is capable of revealing significantly more bugs than the baseline when testing with one test case. The test efficiency of MORTAR is significantly higher than the baseline.

3) *Robustness*: When testing different dialogue systems, METAL's Rate⁺ dropped by 57.7 percentage points (pp), from 86.3% on DS1 to 28.6% on DS6). As a comparison, MORTAR's Rate⁺ dropped by 51.8 pp, from 86.5% on DS1 to 34.7% on DS6. The 10.2% lower performance drop indicates better effectiveness and performance consistency of MORTAR. In general, it is observed that MORTAR has better robustness than the baseline in testing different dialogue systems.

Answer to RQ1: MORTAR is 150% more effective, 153% more efficient and 10.2% more robust than the single-turn MT baseline.

B. RQ2. Characteristics of Detected Bugs

To answer RQ2, we examine the characteristic of bugs in terms of their diversity, precision and uniqueness.

1) *Diversity*: Regarding the diversity of detected bugs, we compare the coverage of L1-Bugs, L2-Bugs and L3-Bugs revealed by MORTAR and METAL. As defined in Section IV-G, L1-Bugs are detected by both MT and RBT with different triggering conditions, L2-Bugs are detected by reusing effective test cases in RBT, and L3-Bugs are detected by reusing ineffective test cases in RBT.

As shown in Table IV, both METAL and MORTAR are able to cover bugs from all three types. However, the number

and proportion of bugs detected varies for the two approaches. MORTAR reveals more than three times as many L3-Bugs as METAL, and the proportion of L3-Bugs among all detected bugs is also 28.6% higher with MORTAR (0.459% vs. 0.357% for MORTAR and METAL, respectively). An analysis of the dispersion in the number of detected bugs across the three bug types shows that MORTAR exhibits the CV that is 1.4% lower than that of METAL. This suggests that MORTAR achieves a more balanced detection performance across different bug categories. Overall, MORTAR achieves greater bug diversity compared to the baseline and detects 200% more L3-Bugs than the baseline.

2) *Precision*: To measure precision, we randomly sample 100 bugs from the set of all bugs revealed by MORTAR and 100 bugs by METAL. Each sampled bug is then manually labelled as either a true positive or a false positive detection. The manual labelling is conducted by two authors. A detection is classified as false positive if both evaluators: a) consider the corresponding question to be clear and understandable, and b) assess the dialogue system's output as factually correct and incorrectly flagged as a bug. The two labellers independently score the dialogue system output in each sample with a number ranging from zero to ten, a high score indicates the correct output and a false positive in testing. We classify cases with a score below six as true positive detections. After independently scoring the samples, the two evaluators discuss uncertain cases and, where possible, reach a consensus on the final decision.

As described in Section IV-G, PPD is calculated as the number of true positive detections divided by the number of positive detections, measuring the precision of detection. According to the manual check results, the PPD of METAL and MORTAR are 45.5% and 70.5% respectively, indicating MORTAR has a 54.9% higher precision than METAL.

In addition, the Cohen's Kappa of METAL and MORTAR are 0.778 and 0.832 respectively, indicating a higher level of inter-rater agreement on MORTAR's detection outcomes. The lower Kappa score for METAL may reflect a greater degree of unresolved disagreement between the two evaluators. Further discussion of false positive detections is provided in Section V-D2. Overall, MORTAR demonstrates a higher proportion of true positive detections, indicating superior precision compared to the baseline.

3) *Uniqueness*: To analyse the uniqueness of bugs, we measure the overlap of questions that reveal bugs. As defined in Section IV-G, in MORTAR and METAL, if a bug is detected with the same question, it will be considered as overlapping. Otherwise, the MT method will be credited for the uniquely revealed bug, which is not discovered by the other method.

The result of uniqueness evaluation is presented in Table V. On average, 35.6% of the bugs revealed by MORTAR are unique, representing a 37.5% higher ratio compared to METAL. This ratio is computed using the full set of 500 test seeds for METAL and 403 test seeds for MORTAR. When evaluated on the common subset of 403 test seeds, which is shown in brackets for METAL column, the ratio of unique bugs detected by MORTAR exceeds that of METAL by more than a factor of 4.4.

Notably, MORTAR demonstrates the strongest performance

TABLE V

THE RATIO OF UNIQUE BUGS REVEALED BY METAL AND MORTAR. HIGHER RATIO INDICATES MORE UNIQUE BUGS ARE REVEALED BY THE TESTING METHOD. FOR METAL, THE RATIO IN BRACKETS IS THE UNIQUE BUGS REVEALED USING THE SAME TEST SEEDS WITH MORTAR.

DS	METAL				MORTAR			
	L1-Bugs	L2-Bugs	L3-Bugs	Overall	L1-Bugs	L2-Bugs	L3-Bugs	Overall
DS1	20.5% (1.0%)	22.9% (2.7%)	0% (0%)	20.9% (1.2%)	25.6%	29.5%	0%	26.1%
DS2	20.6% (1.3%)	27.6% (7.8%)	0% (0%)	22.3% (2.9%)	30.0%	35.3%	0%	31.2%
DS3	20.2% (1.5%)	32.1% (17.9%)	35.5% (13.0%)	22.7% (5.0%)	28.4%	47.9%	65.1%	32.9%
DS4	21.5% (2.6%)	38.0% (22.6%)	0% (0%)	25.5% (7.5%)	30.9%	35.8%	0%	31.8%
DS5	22.9% (4.9%)	43.6% (30.4%)	53.9% (38.8%)	31.1% (14.9%)	31.8%	57.4%	62.0%	42.1%
DS6	24.4% (5.5%)	49.8% (38.9%)	63.4% (43.5%)	34.6% (18.8%)	36.5%	71.6%	77.0%	53.0%
Mean	21.6% (2.7%)	35.2% (19.4%)	24.2% (14.9%)	25.9% (8.0%)	30.3%	45.4%	32.6%	35.6%

TABLE VI

THE RESULT OF MORTAR’S DIALOGUE-LEVEL PERTURBATIONS IN TEST CASE GENERATION AND THEIR USAGE IN MR VIOLATION DETECTIONS.

	DRS	DRR	DRD	DSR	DSD	Sum
Test cases	403	403	403	403	403	2015
Total questions	6,423	4,598	7,625	4,598	7,625	30,869
Unanswerable questions	48	35	77	35	111	306
MR1	6,315	4,539	7,548	4,510	7,473	30,385
MR2	60	24	0	53	41	178
MR3	6,194	4,426	6,194	4,431	6,194	6,194
MR4	151	114	151	116	151	151

on DS6. According to Table V-A, DS6 shows a superior multi-turn dialogue and reasoning performance advantage among all dialogue systems, with the fewest violations of MR observed. An average of 53% of bugs revealed by MORTAR are unique. Among these, 77% are classified as L3-Bugs, suggesting that MORTAR exerts greater testing pressure on DS6 and is more effective in uncovering unique failures when compared to METAL.

Answer to RQ2: The bugs revealed by MORTAR are 1.4% more diverse and MORTAR is 54.9% more precise in revealing bugs. In addition, MORTAR reveals 37.5% more unique bugs than the baseline.

C. RQ3. Ablation Study

In MORTAR, five dialogue-level perturbations are adopted to generate metamorphic test cases, and four MRs are used to form multi-turn metamorphic testing for dialogue systems. This RQ focuses on the contribution of each component to the overall effectiveness in test case generation and bug detection.

Regarding test case generation and perturbation-MR matching, the results are shown in Table VI. It is observed that all MRs contribute to the overall effectiveness. MR2 and MR4 exhibit fewer involvement due to their more stringent conditions, while MR1 accounts for the largest share of bug detections. Overall, each perturbation and MR plays a critical role in test case generation of MORTAR.

Regarding the contribution of perturbations and MRs in testing, we measure the involvement of perturbations and MRs that contribute to bug detections. For MR1 and MR2, the violation detection is contributed by different perturbations individually. Therefore, in a row, the total violation detection of an MR is the sum of the usage of each perturbation. For MR3 and MR4, the violation detection is conducted within

TABLE VII

CONTRIBUTION OF MORTAR’S PERTURBATION IN REVEALING DIALOGUE SYSTEM BUGS. THE RATIO IN THE BRACKET INDICATES THE PROPORTION OF BUGS THAT ARE DISCOVERED WITH CORRESPONDING PERTURBATION IN EACH DIALOGUE SYSTEM. THE BOLDED RESULTS ARE THE HIGHEST PROPORTION OF EACH PERTURBATION’S USAGE IN REVEALING DIFFERENT DIALOGUE SYSTEM BUGS.

DS	DRS	DRR	DRD	DSR	DSD
DS1	11,380 (35.6%)	7,110 (22.3%)	8,494 (26.6%)	4,155 (13.0%)	6,835 (21.4%)
DS2	10,369 (37.3%)	6,296 (22.6%)	7,584 (27.3%)	3,604 (13.0%)	5,878 (21.1%)
DS3	8,433 (34.2%)	4,864 (19.7%)	6,414 (26.0%)	3,511 (14.3%)	5,799 (23.5%)
DS4	7,479 (32.6%)	4,499 (19.6%)	6,003 (26.2%)	3,448 (15.0%)	5,322 (23.2%)
DS5	6,775 (39.2%)	3,807 (22.1%)	4,262 (24.7%)	2,599 (15.1%)	3,920 (22.7%)
DS6	5,200 (40.6%)	2,623 (20.5%)	3,042 (23.8%)	1,856 (14.5%)	3,160 (24.7%)
Mean	8,273 (36.1%)	4,866 (21.3%)	5,966 (26.1%)	3,196 (14.0%)	5,152 (22.5%)

TABLE VIII

CONTRIBUTION OF MORTAR’S PERTURBATION IN REVEALING DIALOGUE SYSTEM BUGS. THE HIGHER VALUE INDICATES MORE BUGS ARE REVEALED USING THE MR WHEN TESTING THE DIALOGUE SYSTEM.

DS	MR1	MR2	MR3	MR4
DS1	25,903 (81.1%)	7 (0.02%)	5,966 (18.7%)	66 (0.2%)
DS2	21,861 (78.6%)	6 (0.02%)	5,844 (21.0%)	88 (0.3%)
DS3	20,242 (82.2%)	5 (0.02%)	4,275 (17.4%)	112 (0.5%)
DS4	19,074 (83.2%)	13 (0.06%)	3,692 (16.1%)	140 (0.6%)
DS5	13,153 (76.2%)	12 (0.07%)	3,975 (23.0%)	124 (0.7%)
DS6	9,722 (75.9%)	7 (0.05%)	2,956 (23.1%)	120 (0.9%)
Mean	18,326 (80.0%)	8 (0.04%)	4,451 (19.4%)	108 (0.5%)

the group of different versions of each original question (see Section III-D), such that each perturbation may contribute multiple times or is not involved in some MR violation detection, and we only count the number of groups.

As shown in Table VII, all perturbations are useful in bug detection, with each responsible for over 13.0% of identified bugs. Among them, perturbation DRS (Dialogue Round Shuffle) is the most effective, accounting for over one-third of all bug discoveries. If one perturbation is excluded in MORTAR, the performance drop can be estimated with the ratio of bugs that are discovered with the corresponding perturbation in testing. It is observed that all perturbations in MORTAR contribute to the overall effectiveness when testing different dialogue systems.

To measure the contribution of different MRs, we count the number of different MR violations. The result is shown in Table VIII. It is observed that all MRs are generally effective in testing. The most effective MR is MR1, which contributes to 80.0% of bugs. Affected by the relatively smaller number of violation detections, MR2 and MR4 do not reveal a large number of bugs in testing.

To provide more insights into the effectiveness of MRs, we further analyse the type of bugs revealed by all MRs. According to the results shown in Table IX, in the bugs revealed by MR1, 78.2% are L1-Bugs, 21.4% are L2-Bugs, and only 0.4% are L3-Bugs. As a comparison, 52% of bugs revealed using MR2 are L2-Bugs, and 1.1% of bugs revealed by MR4 are L3-Bugs. MR2 has the highest portion of L2-Bugs among all bugs discovered by MR2. Although MR2 has a relatively lower number of violations, it has outstanding performance in revealing L2-Bugs, and this brings unique value in testing. A similar phenomenon is observed for MR4

TABLE IX
THE PROPORTION OF DIFFERENT TYPES OF BUGS REVEALED BY DIFFERENT MRS IN MORTAR. WE BOLD THE RESULTS TO EMPHASIZE THE HIGHEST DISCOVERY EFFICIENCY OF SPECIFIC BUG TYPE USING DIFFERENT MRS.

MRs	L1-Bugs	L2-Bugs	L3-Bugs
MR1	78.2%	21.4%	0.4%
MR2	48.0%	52.0%	0.0%
MR3	66.9%	32.4%	0.7%
MR4	51.4%	47.5%	1.1%

and L3-Bugs. In general, we argue that all perturbations and MRs are effective and contribute to the overall effectiveness from different aspects.

Answer to RQ3: All perturbations contribute to the effectiveness of test case generation and bug discovery, with every perturbation accounting for more than 13% of the overall bug detection performance. All MRs have varied contributions to the discovery of bugs. The least effective MR has higher discovery efficiency for L2-Bugs and L3-Bugs. Each component in MORTAR contributes to the overall effectiveness when testing LLM-based dialogue systems.

D. Discussion

1) *On the unsuccessful information extraction:* Regarding the observed unstable behaviour of unsuccessful information extraction in test case generation of MORTAR, after investigation, the failed extraction dialogues have 92.8% overlap with effective original test cases of DS6 in RBT. DS6 is the best-performing dialogue system among the test objects. In most failed cases, the LLM cannot follow the instructions in prompts that require JSON-formatted output, even with multiple retries with higher output temperatures. In other cases, it is observed that the entities in these dialogues are usually complicated with entities in specific domains, e.g. team name abbreviations in sports. We anticipate that the complexity of these dialogue systems exceeds the information extraction processing capacity of MORTAR’s LLM-based IE pipelines, showing the potential direction for future improvement.

2) *On the false positive detections:* The most common false positive detections are overlong answers. For example, the short answer “Yes” is expected in yes-no questions, the prompt also requests the answer to be concise, exact and short. As some questions might require reasoning or analysis, the DS tend to explain the answer with overlong paragraphs, as a result, the semantic similarity is lower than the threshold. The labellers report that some reasonable wordy outputs of dialogue systems could be regarded as false positive detections, e.g. the expected answer: “Yes” and the output: “Yes they did” do not have high semantic similarity, but the output is actually acceptable. Nevertheless, considerable outputs exceed the necessary explanation and give exceedingly long paragraphs, they are regarded as true positive detections. We expect future improvement in semantic similarity measurement to realise lower false positive rates.

Besides, during manual check, both participants reported that METAL’s perturbation brings too much noise to the original question, and considerable questions’ semantics have been altered, which contradicts the expectation. For example, in multi-turn testing with the theme of biology, a question “What is a gene?” is perturbed with *add random word* perturbation in METAL, and the perturbed question “What is Pear a gene?” significantly changed the semantics of the sentence. The dialogue system uses its internal knowledge to answers this question with explanations about the genes in fruits. It is worth noting that we have already constrained METAL not to perform semantic-altering perturbations. If the constraint is removed, the precision of METAL is expected to be lower. The Kappa of METAL manual check result (0.778) is lower than MORTAR’s (0.832), this could be caused by more disagreement on this issue in METAL.

The false positive bugs in MORTAR are different from METAL. As MORTAR does not directly change the form of question but changes the context, MORTAR’s false positive detections mainly come from extremely short questions. For example, in a perturbed test case, the question “How?” is asked after “Did the movie break any records?”. But in the original test case, “How?” follows question “When did Kyle die?”. The target information of question “How?” is changed in the perturbed test case, but the equivalent context check failed to indicate this change as the original previous question “When did Kyle die?” has been asked earlier in the perturbed test case, thus the necessary information to answer the question “How?” is mistakenly regarded as unchanged.

It can be concluded from the false positive bug analysis that single-turn perturbation can not guarantee semantic preserving in multi-turn testing scenarios as short questions are very common. For dialogue-level perturbations, the performance of the equivalent context check is the main weakness of MORTAR. Future work is expected to further improve the overall performance of MORTAR in multi-turn testing.

3) *On the performance evaluation of dialogue systems:* Our experiments are conducted on six dialogue systems that are based on different LLMs in terms of source and parameter size. DS1, DS2 and DS3 are based on LLMs of the same source but different parameter sizes, they form the comparison group 1. DS3, DS4, DS5 and DS6 are based on different sourced models with similar parameter sizes (7-9 billion), they form the comparison group 2.

It is well-acknowledged that larger language models are capable of better memory of knowledge in the training dataset and resulting in better reasoning capability [46]. However, according to the test results in Table V-A, LLM with a larger parameter size exhibits higher proportions of violations to MR2 and MR4. This contradicts the trend of increased reasoning capability as expectations. It is anticipated that the training datasets of these LLMs are contaminated by the dataset selected in this study. MORTAR is capable of reusing this dataset and building multi-turn metamorphic testing to reveal new bugs. Additionally, we anticipate the integration of MORTAR in model training is also beneficial to avoid over-fitting and improve the generalisation in real-world applications.

Regarding the dialogue capability of dialogue systems, most dialogue systems perform worse in multi-turn MT. In Table V-A, dialogue systems generally exhibit higher Rate⁺ in MORTAR than METAL. Specifically, the increment of DS6 is 21.3%, indicating DS6 is potentially more susceptible to altered context, although it is already the best candidate among all dialogue systems with the least error rate in the performance evaluation. For other dialogue systems, it is anticipated that their performance is unsatisfactory that even the positive rate of RBT is already relatively high, therefore MORTAR is generally replicating existing bugs in RBT with different triggering conditions, and the incremental effectiveness of multi-turn testing is relatively lower. This anticipation is supported by the trend of the number of different levels of bugs, which is shown in Table IV, the number of L1-Bugs descends from DS1 to DS6, and the number of L2-Bugs and L3-Bugs revealed by MORTAR generally ascends from DS1 to DS6.

MORTAR is more effective in revealing bugs in multi-turn testing for dialogue systems, but this does not diminish the practical value of METAL in testing, as the test datasets and methods are usually limited. Considering the constraints in real-world applications, it is ideal to build LLM-based dialogue systems using larger models, and to conduct comprehensive testing to discover various unexpected behaviours to better ensure the service quality.

VI. THREATS TO VALIDITY

A. Internal Threats

The dialogue-level perturbations rely on randomness to generate follow-up test cases, which lack fine-grained control in generation.

As there does not exist any plug-and-play tool that is primarily designed for the purpose of equivalent context checking, we crafted a feasible checking process to automate MORTAR. The implemented equivalent context check may not be able to fully and accurately acquire the equivalence of context. In practical testing, over 80% of the original dataset is successfully processed, realising a precision of 70+% in bug detection of MORTAR. There exists a large space for performance improvements. Adaptations of other natural language processing methods might also fit into the setting of this task and achieve better performance.

B. External Threats

Given the limited availability of multi-turn test datasets, we only use the open-domain multi-turn reading comprehension dataset CoQA as the test seed. There might exist multiple issues: the dataset might contain mislabelled answers, and the equivalent context check in MORTAR may not function as expected in some original test cases. For the broader dialogue system testers, these issues might persist for their domain-specific datasets. Not all multi-turn datasets can be processed by MORTAR without adjustments in equivalent context check process which require domain-specific knowledge to handle. We argue that the feasibility of the framework of MORTAR in open-domain multi-turn testing is validated, and we endeavour

to improve the generalisation of MORTAR in domain-specific dialogue systems in the future.

Limited by computing capacity, we are unable to exhaustively test all LLM-based dialogue systems. The dialogue systems in this research are based on six popular LLMs, which are potentially the basis of considerable dialogue systems in real-world usage for their relatively light-weight computing capacity requirements. A broader testing on more different types and parameter sizes of LLMs is believed to benefit both the industry and academia.

We are unable to provide fixing approaches to improve the multi-turn dialogue performance of LLM-based dialogue systems. It is meaningful yet out of the scope of this research.

VII. RELATED WORK

A. Testing Dialogue Systems

Current dialogue test cases are generally from human-generated datasets and LLM-generated datasets. SQuAD and SQuAD 2.0 are classic single-turn QA datasets that have been widely used for training and testing of QA systems [47], [48]. CoQA dataset is a free-form multi-turn conversational QA dataset with unanswerable questions [18]. MuTual is a multi-turn reasoning-based dialogue dataset in open-domain [49]. MT-Bench is a 2-turn open-end dialogue dataset that is used to evaluate chatbot’s conversation performance [11]. The “LLM-as-a-judge” approach is reported as feasible when comparing strong LLM’s alignment with human preference. Later, using LLM to generate test cases become a recent trend [10], [26]–[28]. However, it is worth noting that the “LLM-as-a-judge” approach comes with intrinsic shortcomings e.g., self-enhancement [11], bias [50]. Besides, training data contamination is receiving increasing attention [25]. The area calls for more complete and reliable testing for dialogue systems.

B. Information Extraction

Information extraction (IE) is a classic task of natural language processing. Many downstream applications undertake IE before sophisticated pattern analysis. The extracted information can be used for dialogue management [51], test case generation [22], summarisation [40], and many other tasks. It is observed that LLM-based IE approaches are capable of more flexibility and better extraction performance [41], [52]. In this paper, we use an LLM-based IE approach to extract the dialogue question’s context reliance and tag the context equivalence of questions in perturbed test cases.

C. Metamorphic Testing

To alleviate the oracle problem in software testing, MT was first introduced in 1998 [13]. Later and up to now, MT is commonly used in both traditional software systems [53], [54] and machine learning systems [14], [55], [56]. The unique value of MT is that it could be used to handle low-resource testing scenarios [32], and has the potential for testing GenAI systems [2]. Specifically, METAL uses utterance perturbations and MR templates to evaluate LLMs [16]. Ontology-based MT provides concrete task-oriented test cases but falls

short in generalisation. [57]. KGIT uses knowledge graphs as test seeds and builds a metamorphic testing-based scheme to implement large-scale inference tests [58]. Drowzee uses metamorphic testing and constrained logic programming to detect hallucinations in LLMs [22]. DialTest uses a set of MRs to effectively generate test cases for dialogue systems [35]. Existing MT approaches lack systematic analysis and usage of dialogue-level perturbations and multi-turn dialogue test case generation. Multi-turn dialogue testing remains challenging.

VIII. CONCLUSION AND FUTURE WORKS

LLM-based dialogue systems are now widely used in real-world applications. While there exist many single-turn testing methods for them, they are limited to the single-turn scenarios, leaving the real-world multi-turn usage scenario significantly underexplored. In this research, a multi-turn metamorphic testing approach, MORTAR, is proposed to mitigate the oracle problem in dialogue system testing. In MORTAR, a series of formalised MRs and dialogue-level perturbations are proposed and implemented with the multi-turn MT framework to generate follow-up test cases and reveal bugs in LLM-based dialogue systems. Given an automated MR matching mechanism, MORTAR can be fully automated without human interference. The test results show MORTAR is more effective than the most effective single-turn metamorphic testing baseline. MORTAR detects over 150% more bugs than the single-turn MT baseline, and over one-third of them are unique bugs. The revealed bugs are of higher quality when compared with the baseline in terms of diversity, precision and uniqueness. The component contribution analysis further validates the effectiveness of components in MORTAR. In general, MORTAR is the first multi-turn metamorphic testing approach for LLM-based dialogue systems, which exhibits satisfactory performance in testing.

We wish to point out that (i) these 4 types of MRs are not restricted to the 5 types of perturbations introduced in this paper, and (ii) different application domains for the multi-turn dialogue systems may have different MRs as their characterisations. MORTAR is expected to inspire both dialogue system developers and researchers to build more comprehensive testing methods for dialogue systems. In the future, we intend to refine the equivalent context check process and reduce the false positive detections. The semantic similarity measurement can also be enhanced to improve the accuracy of judgment.

REFERENCES

- [1] A. Algherairy and M. Ahmed, "A review of dialogue systems: current trends and future directions," *Neural Computing and Applications*, vol. 36, no. 12, pp. 6325–6351, 2024.
- [2] A. Aleti, "Software testing of generative ai systems: Challenges and opportunities," in *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, 2023, pp. 4–14.
- [3] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, "Hotpotqa: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018, pp. 2369–2380.
- [4] B. Wang, W. Chen, H. Pei, C. Xie, M. Kang, C. Zhang, C. Xu, Z. Xiong, R. Dutta, R. Schaeffer *et al.*, "Decodingtrust: A comprehensive assessment of trustworthiness in gpt models," in *NeurIPS*, 2023.
- [5] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," *arXiv preprint arXiv:2009.03300*, 2020.
- [6] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman, "Gpqa: A graduate-level google-proof q&a benchmark," *arXiv preprint arXiv:2311.12022*, 2023.
- [7] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang *et al.*, "A survey on evaluation of large language models," *ACM Transactions on Intelligent Systems and Technology*, vol. 15, no. 3, pp. 1–45, 2024.
- [8] ShareGPT, "ShareGPT Data," 2023, accessed: 2024-10-29. [Online]. Available: https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered
- [9] N. Li, Z. Han, I. Steneker, W. Primack, R. Goodside, H. Zhang, Z. Wang, C. Menghini, and S. Yue, "Llm defenses are not robust to multi-turn human jailbreaks yet," *arXiv preprint arXiv:2408.15221*, 2024.
- [10] G. Bai, J. Liu, X. Bu, Y. He, J. Liu, Z. Zhou, Z. Lin, W. Su, T. Ge, B. Zheng, and W. Ouyang, "MT-bench-101: A fine-grained benchmark for evaluating large language models in multi-turn dialogues," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 7421–7454. [Online]. Available: <https://aclanthology.org/2024.acl-long.401>
- [11] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing *et al.*, "Judging llm-as-a-judge with mt-bench and chatbot arena," *Advances in Neural Information Processing Systems*, vol. 36, pp. 46 595–46 623, 2023.
- [12] H. Huang, Y. Qu, H. Zhou, J. Liu, M. Yang, B. Xu, and T. Zhao, "On the limitations of fine-tuned judge models for llm evaluation," 2024. [Online]. Available: <https://arxiv.org/abs/2403.02839>
- [13] F. Chan, T. Y. Chen, S. C. Cheung, M. F. Lau, and S.-M. Yiu, "Application of metamorphic testing in numerical analysis," in *Proceedings of the IASTED International Conference on Software Engineering (SE'98)*, 1998.
- [14] S. Chen, S. Jin, and X. Xie, "Testing your question answering software via asking recursively," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2021, pp. 104–116.
- [15] Q. Shen, J. Chen, J. M. Zhang, H. Wang, S. Liu, and M. Tian, "Natural test generation for precise testing of question answering software," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–12.
- [16] S. Hyun, M. Guo, and M. A. Babar, "Metal: Metamorphic testing framework for analyzing large-language model qualities," in *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2024, pp. 117–128.
- [17] J. Gao, M. Galley, and L. Li, "Neural approaches to conversational ai," in *The 41st international ACM SIGIR conference on research & development in information retrieval*, 2018, pp. 1371–1374.
- [18] S. Reddy, D. Chen, and C. D. Manning, "Coqa: A conversational question answering challenge," *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 249–266, 2019.
- [19] Y. Fan and X. Luo, "A survey of dialogue system evaluation," in *2020 IEEE 32nd International Conference on Tools with Artificial Intelligence (ICTAI)*. IEEE, 2020, pp. 1202–1209.
- [20] Z. Guo, R. Jin, C. Liu, Y. Huang, D. Shi, L. Yu, Y. Liu, J. Li, B. Xiong, D. Xiong *et al.*, "Evaluating large language models: A comprehensive survey," *arXiv preprint arXiv:2310.19736*, 2023.
- [21] S. Mehri, M. Eric, and D. Hakkani-Tur, "Dialoglue: A natural language understanding benchmark for task-oriented dialogue," *arXiv preprint arXiv:2009.13570*, 2020.
- [22] N. Li, Y. Li, Y. Liu, L. Shi, K. Wang, and H. Wang, "Drowzee: Metamorphic testing for fact-conflicting hallucination detection in large language models," *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA2, Oct. 2024. [Online]. Available: <https://doi.org/10.1145/3689776>
- [23] Z. Li, X. Xu, T. Shen, C. Xu, J.-C. Gu, Y. Lai, C. Tao, and S. Ma, "Leveraging large language models for nlg evaluation: Advances and challenges," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 16 028–16 045. [Online]. Available: <https://aclanthology.org/2024.emnlp-main.896/>
- [24] J. Ou, J. Lu, C. Liu, Y. Tang, F. Zhang, D. Zhang, and K. Gai, "Dialogbench: Evaluating llms as human-like dialogue systems," *arXiv preprint arXiv:2311.01677*, 2023.

- [25] I. Mirzadeh, K. Alizadeh, H. Shahrokhi, O. Tuzel, S. Bengio, and M. Farajtabar, "Gsm-symbolic: Understanding the limitations of mathematical reasoning in large language models," *arXiv preprint arXiv:2410.05229*, 2024.
- [26] Y. Sun, C. Liu, K. Zhou, J. Huang, R. Song, W. X. Zhao, F. Zhang, D. Zhang, and K. Gai, "Parrot: Enhancing multi-turn instruction following for large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 9729–9750.
- [27] W.-C. Kwan, X. Zeng, Y. Jiang, Y. Wang, L. Li, L. Shang, X. Jiang, Q. Liu, and K.-F. Wong, "Mt-eval: A multi-turn capabilities evaluation benchmark for large language models," *arXiv preprint arXiv:2401.16745*, 2024.
- [28] X. Wang, Z. Wang, J. Liu, Y. Chen, L. Yuan, H. Peng, and H. Ji, "Mint: Evaluating llms in multi-turn interaction with tools and language feedback," *arXiv preprint arXiv:2309.10691*, 2023.
- [29] H. Duan, J. Wei, C. Wang, H. Liu, Y. Fang, S. Zhang, D. Lin, and K. Chen, "BotChat: Evaluating LLMs' capabilities of having multi-turn dialogues," in *Findings of the Association for Computational Linguistics: NAACL 2024*, K. Duh, H. Gomez, and S. Bethard, Eds. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 3184–3200. [Online]. Available: <https://aclanthology.org/2024.findings-naacl.201>
- [30] S. Segura, G. Fraser, A. B. Sanchez, and A. Ruiz-Cortés, "A survey on metamorphic testing," *IEEE Transactions on software engineering*, vol. 42, no. 9, pp. 805–824, 2016.
- [31] T. Y. Chen, S. C. Cheung, and S. M. Yiu, "Metamorphic testing: a new approach for generating next test cases," *arXiv preprint arXiv:2002.12543*, 2020.
- [32] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. Tse, and Z. Q. Zhou, "Metamorphic testing: A review of challenges and opportunities," *ACM Computing Surveys (CSUR)*, vol. 51, no. 1, pp. 1–27, 2018.
- [33] J. M. Zhang, M. Harman, L. Ma, and Y. Liu, "Machine learning testing: Survey, landscapes and horizons," *IEEE Transactions on Software Engineering*, vol. 48, no. 1, pp. 1–36, 2020.
- [34] Q. Shen, J. Chen, J. M. Zhang, H. Wang, S. Liu, and M. Tian, "Natural test generation for precise testing of question answering software," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–12.
- [35] Z. Liu, Y. Feng, and Z. Chen, "Dialtest: automated testing for recurrent-neural-network-driven dialogue systems," in *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2021, pp. 115–126.
- [36] K. Tu, M. Jiang, and Z. Ding, "A metamorphic testing approach for assessing question answering systems," *Mathematics*, vol. 9, no. 7, p. 726, 2021.
- [37] X. Zhang, C. Li, D. Yu, S. Davidson, and Z. Yu, "Filling conversation ellipsis for better social dialog understanding," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 05, 2020, pp. 9587–9595.
- [38] R. Mitkov, *The Oxford handbook of computational linguistics*. Oxford university press, 2022.
- [39] R. Sukthankar, S. Poria, E. Cambria, and R. Thirunavukarasu, "Anaphora and coreference resolution: A review," *Information Fusion*, vol. 59, pp. 139–162, 2020.
- [40] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph rag approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.
- [41] B. Zhang and H. Soh, "Extract, define, canonicalize: An llm-based framework for knowledge graph construction," *arXiv preprint arXiv:2404.03868*, 2024.
- [42] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou, C. Li, C. Li, D. Liu, F. Huang *et al.*, "Qwen2 technical report," *arXiv preprint arXiv:2407.10671*, 2024.
- [43] Mistral, "Mistral-7b-instruct-v0.3 model," 2024. [Online]. Available: <https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3>
- [44] AI@Meta, "Llama 3 model card," 2024. [Online]. Available: https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md
- [45] G. Team, "Gemma," 2024. [Online]. Available: <https://www.kaggle.com/m/3301>
- [46] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [47] P. Rajpurkar, "Squad: 100,000+ questions for machine comprehension of text," *arXiv preprint arXiv:1606.05250*, 2016.
- [48] P. Rajpurkar, R. Jia, and P. Liang, "Know what you don't know: Unanswerable questions for squad," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2018, pp. 784–789.
- [49] L. Cui, Y. Wu, S. Liu, Y. Zhang, and M. Zhou, "Mutual: A dataset for multi-turn dialogue reasoning," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 1406–1416.
- [50] H. Huang, Y. Qu, J. Liu, M. Yang, and T. Zhao, "An empirical study of llm-as-a-judge for llm evaluation: Fine-tuned judge models are task-specific classifiers," *arXiv preprint arXiv:2403.02839*, 2024.
- [51] Z. Chen, Y. Liu, L. Chen, S. Zhu, M. Wu, and K. Yu, "Opal: Ontology-aware pretrained language model for end-to-end task-oriented dialogue," *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 68–84, 2023.
- [52] X. Wei, X. Cui, N. Cheng, X. Wang, X. Zhang, S. Huang, P. Xie, J. Xu, Y. Chen, M. Zhang *et al.*, "Zero-shot information extraction via chatting with chatgpt," *arXiv preprint arXiv:2302.10205*, 2023.
- [53] Z. Zhuang, P. Li, P. Ma, W. Meng, and S. Wang, "Testing graph database systems via graph-aware metamorphic relations," *Proceedings of the VLDB Endowment*, vol. 17, no. 4, pp. 836–848, 2023.
- [54] Z. Q. Zhou, S. Xiang, and T. Y. Chen, "Metamorphic testing for software quality assessment: A study of search engines," *IEEE Transactions on Software Engineering*, vol. 42, no. 3, pp. 264–284, 2015.
- [55] X. Xie, J. W. Ho, C. Murphy, G. Kaiser, B. Xu, and T. Y. Chen, "Testing and validating machine learning classifiers by metamorphic testing," *Journal of Systems and Software*, vol. 84, no. 4, pp. 544–558, 2011.
- [56] W. Wang, J.-t. Huang, W. Wu, J. Zhang, Y. Huang, S. Li, P. He, and M. R. Lyu, "Mttm: Metamorphic testing for textual content moderation software," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2387–2399.
- [57] J. Božić, "Ontology-based metamorphic testing for chatbots," *Software Quality Journal*, vol. 30, no. 1, pp. 227–251, 2022.
- [58] J. Wang, Y. Li, Z. Chen, L. Chen, X. Zhang, and Y. Zhou, "Knowledge graph driven inference testing for question answering software," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.